

DOCUMENTAZIONE TECNICA

Sistema di Controllo per Braccio Robotico IoT

Progetto IoT con Architettura MQTT

Hardware	ESP32-S3 DevKitC-1
Attuatori	4x Servo Motor (Spalla, Gomito, Polso, Claw)
Protocollo	MQTT 3.1.1 + WebSocket/TLS
App Mobile	Android (Kotlin + Jetpack Compose)
Simulazione 3D	Three.js / WebGL in WebView
Broker	Eclipse Mosquitto (Docker + Ngrok)
Manifattura	FreeCAD + Stampa 3D PLA

Indice dei Contenuti

Indice dei Contenuti	2
1. Introduzione e Obiettivi del Progetto	4
1.1 Panoramica Generale	4
1.2 Obiettivi Tecnici	4
1.3 Stack Tecnologico	4
2. Architettura del Sistema	5
2.1 Visione d'Insieme	5
2.2 Topologia di Rete	5
2.3 Flusso Dati End-to-End	5
2.4 Protocollo di Comando	5
2.5 Considerazioni sulla Latenza	5
3. Hardware: ESP32-S3 e Attuatori	7
3.1 Microcontrollore ESP32-S3 DevKitC-1	7
3.2 Mappatura Pin GPIO e Canali LEDC	7
3.3 Parametri PWM e Calibrazione	7
3.4 Servo-Motori e Caratteristiche Meccaniche	8
4. Firmware ESP32-S3 (PlatformIO / Arduino)	9
4.1 Ambiente di Sviluppo: PlatformIO	9
4.2 Architettura del Firmware	9
4.2.1 Struttura Dati ServoState	9
4.2.2 Callback MQTT e Validazione	9
4.2.3 Routine di Interpolazione Asincrona	10
4.3 Gestione della Connettività	10
4.3.1 WiFi	10
4.3.2 Client MQTT (PubSubClient)	10
5. Infrastruttura Broker MQTT	12
5.1 Eclipse Mosquitto su Docker	12
5.2 Configurazione Dual-Listener	12
5.3 Tunnel Ngrok: Architettura Dual-Endpoint	12
5.4 Script di Status Globale	13
5.5 Sicurezza e Autenticazione	13
6. Applicazione Android (Kotlin + Jetpack Compose)	14
6.1 Architettura Ibrida Nativa/Web	14
6.2 Client MQTT Asincrono	14
6.3 Simulazione Cinematica 3D (Three.js in WebView)	14
6.3.1 Albero Cinematico	14
6.3.2 Caricamento STL e WebViewAssetLoader	14

6.4 Criticità Architetture Risolte	15
6.4.1 Vertex Normals Mancanti (Black Shading)	15
6.4.2 Disallineamento Pivot-Mesh (CAD Offset).....	15
6.4.3 Mappatura Dominio JNI (0-indexed vs 1-indexed).....	15
6.5 Strato di Debug e Calibrazione	15
7. Interfaccia Web di Controllo	16
7.1 Architettura e Stack	16
7.2 Controllo Vettoriale Continuo (Jogging)	16
7.3 Compatibilità Mobile e Touch.....	16
7.4 Gestione dello Stato degli Angoli	16
8. Progettazione e Manifattura Meccanica	18
8.1 Processo di Design CAD (FreeCAD)	18
8.2 Componenti Progettati	18
8.3 Stampa 3D	18
8.4 Note sulla Configurazione Pinza.....	18
9. Test e Validazione.....	19
9.1 Test del Firmware	19
9.1.1 Test Connettività MQTT	19
9.1.2 Test Interpolazione Cinematica	19
9.2 Test dell'Interfaccia Web.....	19
9.3 Test della Simulazione 3D	19
9.4 Metriche di Prestazione	19
10. Sicurezza del Sistema.....	21
10.1 Modello di Minaccia	21
10.2 Misure Implementate	21
10.3 Limitazioni e Raccomandazioni Future	21
11. Conclusioni e Sviluppi Futuri	22
11.1 Risultati Raggiunti.....	22
11.2 Sviluppi Futuri.....	22
11.2.1 Inverse Kinematics (IK)	22
11.2.2 Controllo a Traiettoria.....	22
11.2.3 Feedback di Posizione	22
11.2.4 Claw Funzionale.....	22
11.2.5 Infrastruttura Produzione	22
11.3 Conclusioni	23

1. Introduzione e Obiettivi del Progetto

1.1 Panoramica Generale

Il presente documento descrive in dettaglio l'architettura, l'implementazione e le soluzioni tecniche adottate nello sviluppo di un sistema di controllo remoto per braccio robotico basato su protocollo MQTT. Il progetto integra componenti hardware, firmware embedded, comunicazione di rete, applicazione mobile nativa e interfaccia web in un'unica piattaforma coerente.

Il sistema consente il controllo in tempo reale di un braccio robotico a 4 gradi di libertà tramite un'applicazione Android con simulazione cinematica 3D, nonché attraverso un'interfaccia web accessibile da browser, il tutto mediato da un broker MQTT ospitato su infrastruttura cloud containerizzata.

1.2 Obiettivi Tecnici

- Realizzare un sistema di controllo real-time con latenza minima per 4 servo-motori indipendenti
- Implementare un motore cinematico interpolato per movimenti fluidi e controllati
- Sviluppare un'applicazione Android nativa con rendering 3D del modello cinematico
- Deployare un broker MQTT accessibile da internet tramite tunnel sicuri
- Progettare e stampare in 3D i componenti meccanici del braccio robotico
- Integrare un'interfaccia web di telemetria e controllo manuale

1.3 Stack Tecnologico

Livello	Tecnologia	Utilizzo
Hardware	ESP32-S3 DevKitC-1	Microcontrollore principale, WiFi, LEDC PWM
Firmware	Arduino / PlatformIO	Motore cinematico, MQTT client, controllo servo
Comunicazione	MQTT 3.1.1 (PubSubClient)	Trasporto comandi robot/servos/cmd
Broker	Eclipse Mosquitto + Docker	Gestione messaggi, autenticazione PBKDF2
Tunnel	Ngrok (TCP + HTTPS/WSS)	Esposizione internet sicura dual-listener
App Android	Kotlin + Jetpack Compose	UI nativa, gestione stato, JNI bridge
Rendering 3D	Three.js / WebGL	Simulazione cinematica forward kinematics
Web UI	HTML/JS + Paho MQTT	Interfaccia di controllo e telemetria
CAD/Manifattura	FreeCAD + Stampa 3D PLA	Progettazione e produzione componenti

2. Architettura del Sistema

2.1 Visione d'Insieme

Il sistema adotta un'architettura distribuita a tre livelli principali: il livello di presentazione (App Android e Web UI), il livello di trasporto e orchestrazione (Broker MQTT su infrastruttura Docker/Ngrok) e il livello di attuazione (Firmware ESP32-S3 con controllo servo).

2.2 Topologia di Rete

La topologia di rete implementa un pattern publish/subscribe tramite MQTT. I client publisher (App Android, Web UI) inviano comandi al topic `robot/servos/cmd` sul broker Mosquitto. L'ESP32-S3, registrato come subscriber sullo stesso topic, riceve i comandi e li esegue in tempo reale.

TOPIC MQTT

`robot/servos/cmd` → Formato payload: "`<indice_servo>,<angolo>`" (es. "`1,90`")

2.3 Flusso Dati End-to-End

1. L'operatore interagisce con l'App Android o la Web UI (jogging continuo o simulazione 3D)
2. Il client MQTT pubblica un messaggio sul topic `robot/servos/cmd` tramite il broker
3. Il broker Mosquitto autentica il client e instrada il messaggio
4. Il messaggio raggiunge l'ESP32-S3 tramite tunnel Ngrok (TCP raw, porta 25629)
5. Il firmware parse il payload, aggiorna lo stato target del servo specificato
6. La routine di interpolazione asincrona nel `loop()` esegue il movimento fluido
7. Il LEDC genera il segnale PWM calibrato sul pin GPIO corrispondente

2.4 Protocollo di Comando

Il protocollo di comando è volutamente semplice e a bassa latenza. Il payload è una stringa ASCII composta da due campi separati da virgola:

```
Formato: "<servo_index>,<angolo_gradi>"
Esempio servo 0 -> 45°: "0,45"
Esempio servo 2 -> 135°: "2,135"
```

Il firmware esegue validazione sul range dell'indice (0-3) e dell'angolo (0-270°), rigettando i comandi fuori bounds con log su seriale.

2.5 Considerazioni sulla Latenza

Segmento	Latenza Stimata
App Android → Broker (WiFi locale)	< 10 ms
Web UI → Broker (WebSocket/TLS)	< 50 ms
Broker → ESP32 (Ngrok TCP tunnel)	20-80 ms (dipendente da rete)
ESP32: parsing + aggiornamento target	< 1 ms
Latenza totale end-to-end (tipica)	50-150 ms

3. Hardware: ESP32-S3 e Attuatori

3.1 Microcontrollore ESP32-S3 DevKitC-1

L'ESP32-S3 è un System-on-Chip Xtensa LX7 dual-core prodotto da Espressif Systems. La variante DevKitC-1 è stata scelta per la disponibilità di GPIO sufficienti, il modulo WiFi 802.11 b/g/n integrato, e le periferiche hardware necessarie per il controllo PWM multi-canale.

Caratteristica	Valore
CPU	Xtensa LX7 dual-core @ 240 MHz
RAM	512 KB SRAM + 8 MB PSRAM (ext.)
Flash	8 MB
WiFi	802.11 b/g/n 2.4 GHz
Bluetooth	BLE 5.0
GPIO	36 pin programmabili
LEDC (PWM)	8 canali timer, risoluzione fino a 14-bit
Tensione operativa	3.3V (tollerante 5V su alcuni pin)
Alimentazione	USB-C 5V via DevKit

3.2 Mappatura Pin GPIO e Canali LEDC

La periferica LEDC (LED Control) dell'ESP32-S3 è utilizzata per generare i segnali PWM a 50Hz necessari per il controllo dei servo-motori. Ogni servo è assegnato a un canale LEDC indipendente per evitare interferenze temporali.

Servo	GPIO Pin	Canale LEDC / Funzione
Servo 0	GPIO 4	LEDC CH0 – Spalla (base rotazione)
Servo 1	GPIO 5	LEDC CH1 – Gomito (flessione)
Servo 2	GPIO 47	LEDC CH2 – Polso (flessione)
Servo 3	GPIO 48	LEDC CH3 – Claw (presa)

3.3 Parametri PWM e Calibrazione

Il controllo servo avviene tramite modulazione PWM a 50 Hz (periodo 20ms) in accordo con lo standard RC. La larghezza dell'impulso determina la posizione angolare del servo nell'intervallo 0°-270°.

Parametro	Valore
Frequenza PWM	50 Hz (periodo 20.000 µs)
Risoluzione duty cycle	14-bit (16.384 step)
Impulso minimo (0°)	500 µs
Impulso massimo (270°)	2500 µs
Range angolare	0° – 270°
Formula duty cycle	$(\text{pulse_us} * 16384) / 20000$

```
// Calcolo duty cycle per un dato angolo (0-270°)
float pulse_us = 500 + (gradi / 270.0) * (2500 - 500);
uint32_t duty = (uint32_t)((pulse_us * 16384.0) / 20000.0);
```

3.4 Servo-Motori e Caratteristiche Meccaniche

Il progetto utilizza 4 servo-motori di tipo standard RC. La spalla (Servo 0) è soggetta al carico maggiore essendo all'articolazione base, pertanto è configurata in modalità bypass cinematico (movimento istantaneo) per massimizzare la coppia di spunto. Gli altri tre servo (gomito, polso, claw) beneficiano dell'interpolazione fluida.

Servo	Profilo Cinematico
Servo 0 – Spalla (GPIO 4)	Istantaneo (SPEED = 0.0°/s) – Bypass per carico pesante
Servo 1 – Gomito (GPIO 5)	60°/s – 270° in 4.5 secondi
Servo 2 – Polso (GPIO 47)	60°/s – 270° in 4.5 secondi
Servo 3 – Claw (GPIO 48)	60°/s – 270° in 4.5 secondi

4. Firmware ESP32-S3 (PlatformIO / Arduino)

4.1 Ambiente di Sviluppo: PlatformIO

Il firmware è sviluppato con il framework Arduino su PlatformIO IDE, scelto per la gestione avanzata delle dipendenze, la configurazione esplicita dell'ambiente target e l'integrazione con sistemi di build moderni.

platformio.ini

```
[env:esp32-s3-devkitc-1] | platform = espressif32@~6.5.0 | board =  
esp32-s3-devkitc-1 | framework = arduino | lib_deps =  
knolleary/PubSubClient@^2.8
```

4.2 Architettura del Firmware

Il firmware è strutturato attorno a tre responsabilità principali: gestione della connettività di rete (WiFi + MQTT), ricezione e validazione dei comandi, e routine di interpolazione cinematica asincrona non bloccante.

4.2.1 Struttura Dati ServoState

Lo stato di ogni servo è mantenuto in una struttura dati dedicata che separa la posizione corrente dalla posizione target, abilitando l'interpolazione fluida indipendente per ciascun attuatore.

```
struct ServoState {  
    float currentAngle;    // Angolo corrente (aggiornato ad ogni step)  
    float targetAngle;    // Angolo di destinazione (iniettato da MQTT)  
    uint32_t lastUpdate;  // Timestamp ultimo aggiornamento (millis())  
};  
ServoState servoData[NUM_SERVOS]; // Array di 4 istanze
```

4.2.2 Callback MQTT e Validazione

La callback `mqtt_callback()` è invocata dalla libreria `PubSubClient` ogni volta che un messaggio arriva sul topic sottoscritto. La validazione comprende il controllo del formato del payload, il range dell'indice servo (0-3) e il clamping dell'angolo nell'intervallo fisico (0-270°).

```
void mqtt_callback(char* topic, byte* payload, unsigned int length) {  
    // Ricostruzione stringa dal buffer byte[]  
    String msg = "";  
    for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];  
  
    int delimiter = msg.indexOf(',');  
    if (delimiter != -1) {  
        int idx    = msg.substring(0, delimiter).toInt();  
        int angle = msg.substring(delimiter + 1).toInt();  
        angle = constrain(angle, 0, 270); // Clamping fisico  
    }  
}
```

```
        if (idx >= 0 && idx < NUM_SERVOS)
            servoData[idx].targetAngle = (float)angle;
    }
}
```

4.2.3 Routine di Interpolazione Asincrona

Il nucleo del motore cinematico risiede nel loop principale e utilizza il pattern delta-time per calcolare incrementi di posizione indipendenti dalla frequenza di esecuzione del loop. Questo garantisce velocità angolare costante indipendentemente dal carico computazionale.

```
void loop() {
    client.loop(); // Processamento stream MQTT (non bloccante)
    uint32_t now = millis();

    for (uint8_t i = 0; i < NUM_SERVOS; i++) {
        float deltaT = (now - servoData[i].lastUpdate) / 1000.0f;
        servoData[i].lastUpdate = now;

        if (abs(servoData[i].targetAngle - servoData[i].currentAngle) >
0.1f) {
            if (SPEED_DEG_PER_SEC[i] == 0.0f) {          // Bypass cinematico
                servoData[i].currentAngle = servoData[i].targetAngle;
            } else {                                       // Interpolazione
fluida
                float step = SPEED_DEG_PER_SEC[i] * deltaT;
                // ... avanzamento con clamping al target
            }
            ledcWrite(LED_CHANNELS[i],
calcolaDuty(servoData[i].currentAngle));
        }
    }
}
```

4.3 Gestione della Connettività

4.3.1 WiFi

La connessione WiFi è gestita tramite la libreria WiFi.h nativa dell'SDK Espressif. Il firmware effettua il binding sulla rete con polling bloccante solo in fase di setup(). In caso di disconnessione durante l'operatività, la funzione reconnect() gestisce il re-handshake MQTT con retry a 3 secondi.

4.3.2 Client MQTT (PubSubClient)

La libreria PubSubClient di Nick O'Leary fornisce un client MQTT 3.1.1 asincrono integrato nel loop Arduino. Il client si connette al broker tramite TCP raw (tunnel Ngrok) con autenticazione username/password. L'identificatore client è generato casualmente ad ogni connessione per evitare conflitti di sessione.

```
String clientId = "ESP32S3-KINEMATICS-" + String(random(0xffff), HEX);  
client.connect(clientId.c_str(), mqtt_user, mqtt_pass);  
client.subscribe(topic_cmd); // "robot/servos/cmd"
```

5. Infrastruttura Broker MQTT

5.1 Eclipse Mosquitto su Docker

Il broker MQTT è deployato come container Docker basato sull'immagine ufficiale eclipse-mosquitto. La scelta di Docker garantisce portabilità, isolamento dell'ambiente e semplicità di gestione del processo con restart policy automatica.

Componente	Configurazione
Immagine Docker	eclipse-mosquitto (latest)
Nome container	mqtt-broker
Restart policy	unless-stopped (demone persistente)
Rete Docker interna	ngrok-net (bridge isolata)
Porte esposte all'host	Nessuna (traffico solo via Ngrok)
Configurazione	/opt/mqtt-ngrok/config/mosquitto.conf
File credenziali	/opt/mqtt-ngrok/config/passwd (PBKDF2)
Dati persistenti	/opt/mqtt-ngrok/data
Log directory	/opt/mqtt-ngrok/log

5.2 Configurazione Dual-Listener

Mosquitto è configurato con due listener indipendenti per supportare contemporaneamente client IoT (ESP32) e client web (browser). Questa configurazione permette di utilizzare il protocollo nativo MQTT su TCP per i microcontrollori e MQTT over WebSocket per i browser, che non possono aprire socket TCP raw.

Listener	Dettagli
Listener 1 – Raw TCP	Porta 1883, bind 0.0.0.0, protocollo MQTT standard
Listener 2 – WebSocket	Porta 9001, bind 0.0.0.0, protocollo MQTT over WS
Autenticazione	Obbligatoria su entrambi (allow_anonymous false)
Hash password	PBKDF2 (mosquitto_passwd)

5.3 Tunnel Ngrok: Architettura Dual-Endpoint

Il container ngrok-agent è configurato tramite ngrok.yml con due tunnel simultanei per esporre entrambi i listener Mosquitto su internet. Il tunnel TCP espone la porta 1883 con un FQDN e porta dinamici allocati nel cloud Ngrok, mentre il tunnel HTTP/HTTPS espone la porta 9001 con TLS automatico.

**Tunnel TCP
(ESP32)**

Endpoint: 6.tcp.eu.ngrok.io:25629 → mqtt-broker:1883 (Raw MQTT per microcontrollori)

**Tunnel
HTTPS/WSS
(Browser)**

Endpoint: *.ngrok-free.dev:443 → mqtt-broker:9001 (MQTT over WebSocket + TLS)

5.4 Script di Status Globale

Lo script `mqtt-status` (installato in `/usr/local/bin/`) interroga le API REST interne dell'agente Ngrok per estrarre dinamicamente gli FQDN e le porte allocate nel cloud, restituendo in `stdout` i parametri di connessione aggiornati per entrambi i tipi di client.

5.5 Sicurezza e Autenticazione

Il sistema implementa un modello di sicurezza a più livelli: autenticazione MQTT obbligatoria con credenziali hashate PBKDF2, traffico web cifrato TLS 1.3 tramite Ngrok, isolamento L4 con nessuna porta esposta direttamente sull'host, e rete Docker interna bridge con accesso controllato solo tra i container autorizzati.

6. Applicazione Android (Kotlin + Jetpack Compose)

6.1 Architettura Ibrida Nativa/Web

L'applicazione Android adotta un'architettura ibrida innovativa: l'interfaccia utente e la logica di controllo sono implementate in Kotlin con Jetpack Compose (framework UI dichiarativo nativo), mentre il motore di rendering 3D è una WebView che ospita un'applicazione Three.js/WebGL. La comunicazione bidirezionale tra layer nativo e web avviene tramite JNI/JavascriptInterface.

Layer	Responsabilità
Kotlin / Compose (Nativo)	State management, UI controls, MQTT client, JNI bridge
WebView / Three.js (Web)	Rendering 3D STL, forward kinematics, debug overlay
JNI JavascriptInterface	Bridge bidirezionale per aggiornamenti cinematici

6.2 Client MQTT Asincrono

L'app implementa un client MQTT asincrono che pubblica comandi di controllo sul topic robot/servos/cmd. Il client è inizializzato con le credenziali del broker e gestisce riconessioni automatiche in caso di perdita del link di rete.

6.3 Simulazione Cinematica 3D (Three.js in WebView)

6.3.1 Albero Cinematico

La simulazione implementa un modello di Forward Kinematics (FK) basato su un albero gerarchico a 4 nodi. Ogni nodo figlio eredita le trasformazioni del nodo padre, simulando il comportamento reale del braccio articolato.

Indice Nodo	Componente	Dettagli
0	Base Statica (BaseArm-Body.stl)	Parent: -1 – Ancora al mondo, non ruota
1	Spalla (Arm2_final.stl)	Parent: 0, Asse Y – Attuatore 0
2	Gomito (Arm3_final.stl)	Parent: 1, Asse X – Attuatore 1
3	Polso / End-Effector (Arm4.stl)	Parent: 2, Asse X – Attuatore 2

6.3.2 Caricamento STL e WebViewAssetLoader

I modelli STL sono serviti alla WebView tramite WebViewAssetLoader, che intercetta le richieste HTTP verso il dominio virtuale `https://appassets.androidplatform.net` e le risolve sugli asset bundlati nell'APK. Questo aggira le restrizioni di sicurezza di Chromium API 30+ che bloccano le richieste XHR verso percorsi `file://`.

```
// Configurazione WebViewAssetLoader
val assetLoader = WebViewAssetLoader.Builder()
    .addPathHandler("/assets/", AssetsPathHandler(context))
    .build()
// STLLoader carica da:
https://appassets.androidplatform.net/assets/model.stl
```

6.4 Criticità Architetture Risolte

6.4.1 Vertex Normals Mancanti (Black Shading)

I file CAD esportati in STL non includono dati di normali per vertice, causando un rendering completamente nero con MeshPhongMaterial. La soluzione implementata esegue il calcolo automatico delle normali a runtime su Three.js prima del calcolo dei bounding box.

```
// Risoluzione black shading su modelli STL CAD
geometry.computeVertexNormals(); // Calcolo normali a runtime
const material = new THREE.MeshPhongMaterial({ color: 0x888888 });
```

6.4.2 Disallineamento Pivot-Mesh (CAD Offset)

I modelli CAD non hanno il fulcro di rotazione centrato sull'origine geometrica, causando rotazioni errate. La soluzione disaccoppia il pivot di rotazione (jointPos) dall'offset della geometria rispetto al fulcro (meshOffset, meshRotation), permettendo compensazioni vettoriali precise per ogni link.

6.4.3 Mappatura Dominio JNI (0-indexed vs 1-indexed)

Gli attuatori fisici nell'array Android sono 0-indexed (0-3), mentre i nodi figlio in Three.js sono 1-indexed (il nodo 0 è la base statica). Un offset logico +1 nella mappatura JNI risolve l'ambiguità senza modificare né il firmware né il rendering.

6.5 Strato di Debug e Calibrazione

L'app implementa uno strato di "Calibrazione Costruttiva" nello state management di Compose che esporta a runtime verso JavaScript i seguenti parametri manipolabili in tempo reale:

- Posizione del Pivot (X, Y, Z) per ogni link
- Offset Traslazionale della Mesh (X, Y, Z) rispetto al pivot
- Rotazione Assoluta della Mesh (Pitch, Yaw, Roll) per correzione CAD
- OrbitControls attivi sulla canvas 3D per ispezione del frustum da qualsiasi angolo

7. Interfaccia Web di Controllo

7.1 Architettura e Stack

L'interfaccia web è una Single Page Application (SPA) vanilla HTML/CSS/JS che non richiede build tools o framework. Si connette al broker MQTT tramite la libreria Paho MQTT su WebSocket con TLS, sfruttando il secondo listener Mosquitto su porta 9001 esposto via tunnel Ngrok HTTPS.

Componente	Implementazione
MQTT Client	Eclipse Paho JavaScript (mqttws31.min.js da CDN)
Trasporto	WebSocket su TLS (WSS) – Ngrok HTTPS endpoint
UI Pattern	Jogging continuo con hold-to-move (mousedown/touchstart)
Stile	Dark theme monocromatico, font monospace

7.2 Controllo Vettoriale Continuo (Jogging)

L'interfaccia implementa un pattern di controllo jogging: invece di inviare un angolo assoluto, l'operatore tiene premuto il pulsante + o - per muovere il servo nella direzione desiderata. Il sistema invia aggiornamenti ogni 100ms finché il pulsante è premuto.

```
// Pattern startMove/stopMove
function startMove(idx, step, event) {
  if (event) event.preventDefault(); // Evita doppio trigger touch
  angles[idx] = Math.max(0, Math.min(270, angles[idx] + step));
  sendUpdate(idx); // Invio immediato al click
  moveInterval = setInterval(() => {
    angles[idx] += step;
    if (angles[idx] >= 270 || angles[idx] <= 0)
      clearInterval(moveInterval);
    sendUpdate(idx); // Aggiornamenti continui ogni 100ms
  }, 100);
}
```

7.3 Compatibilità Mobile e Touch

L'interfaccia è ottimizzata per l'uso su dispositivi touch tramite event listener dedicati: ontouchstart e ontouchend gestiscono gli eventi touch, con event.preventDefault() per sopprimere il comportamento di doppio-click nativo dei browser mobile. La proprietà CSS touch-action: manipulation disabilita lo zoom su doppio tap.

7.4 Gestione dello Stato degli Angoli

Lo stato degli angoli è mantenuto in memoria JavaScript in un array di 4 elementi, inizializzato a 135° (posizione centrale del range 0-270°). Ogni aggiornamento locale dell'array viene immediatamente riflesso sull'etichetta visuale e inviato via MQTT al broker.

```
let angles = [135, 135, 135, 135]; // Posizione iniziale centrata
function sendUpdate(idx) {
    document.getElementById(`val_${idx}`).innerText = angles[idx] + "°";
    const message = new Paho.MQTT.Message(`${idx},${angles[idx]}`);
    message.destinationName = topic_cmd;
    client.send(message);
}
```

8. Progettazione e Manifattura Meccanica

8.1 Processo di Design CAD (FreeCAD)

I componenti meccanici del braccio robotico sono stati progettati interamente in FreeCAD, software CAD open-source parametrico. Il design ha richiesto 7 ore complessive di lavoro, suddivise tra la modellazione dei singoli link, il dimensionamento delle sedi per i servomotori e l'ottimizzazione delle geometrie per la stampa 3D.

8.2 Componenti Progettati

File STL	Componente
BaseArm-Body.stl	Base statica – Struttura di ancoraggio al piano di lavoro
Arm2_final.stl	Link 1 / Spalla – Articolazione di rotazione sull'asse Y
Arm3_final.stl	Link 2 / Gomito – Articolazione di flessione sull'asse X
Arm4.stl	Link 3 / Polso – End-Effector, flessione asse X

8.3 Stampa 3D

I componenti sono stati prodotti tramite stampa FDM (Fused Deposition Modeling) utilizzando filamento PLA (Acido Polilattico). Il PLA è stato scelto per la facilità di stampa, la rigidità meccanica sufficiente per carichi statici moderati, e la disponibilità.

Parametro	Valore
Materiale	PLA (Polylactic Acid)
Processo	FDM (Fused Deposition Modeling)
Tempo totale di design CAD	~7 ore
Tempo totale di stampa	~35 ore
Configurazione pinza	Deprecata (rimossa dall'albero cinematico)

8.4 Note sulla Configurazione Pinza

La configurazione a pinza biforcata originariamente prevista per il Servo 3 (claw) è stata deprecata e rimossa dall'albero cinematico durante lo sviluppo. La decisione è stata presa per semplificare il modello di simulazione 3D e la logica di controllo. Il Servo 3 rimane fisicamente presente e controllabile via MQTT, ma il corrispondente link non è rappresentato nella simulazione Three.js.

9. Test e Validazione

9.1 Test del Firmware

9.1.1 Test Connettività MQTT

La connettività MQTT è stata verificata tramite il monitor seriale di PlatformIO a 115200 baud. Il log mostra chiaramente le fasi di handshake WiFi, il binding TCP con il broker tramite Ngrok, e la ricezione dei comandi.

```
[WIFI] Binding in corso su MERCUSYS_RE_8F7D ....  
[WIFI] Link UP. IP: 192.168.1.XXX  
[MQTT] Handshake TCP in corso... OK  
[CMD] Nuovo target iniettato: Servo 1 -> 90 gradi
```

9.1.2 Test Interpolazione Cinematica

Il comportamento di interpolazione è stato validato osservando il movimento fisico del servo durante l'invio di comandi incrementali. Il profilo a 60°/s garantisce movimenti fluidi e prevedibili, con clamping corretto al raggiungimento del target.

9.2 Test dell'Interfaccia Web

L'interfaccia web è stata testata su browser desktop (Chrome, Firefox) e mobile (Chrome Android). Il pattern jogging con hold-to-move funziona correttamente su entrambi i paradigmi di input, con soppressione del doppio trigger su touch.

9.3 Test della Simulazione 3D

La correttezza della simulazione forward kinematics è stata verificata visivamente utilizzando i parametri di debug esportati dalla app: i pivot sono stati calibrati in modo che la rotazione simulata di ogni link corrisponda al comportamento fisico del braccio. Lo strato di Calibrazione Costruttiva ha permesso di correggere gli offset CAD senza ricompilare il codice.

9.4 Metriche di Prestazione

Metrica	Risultato
Latenza MQTT (LAN)	< 15 ms (app Android → ESP32)
Latenza MQTT (WAN/Web)	40-100 ms (browser → ESP32 via Ngrok)
Frequenza loop ESP32	~1 kHz (non bloccante con client.loop())
Smoothness interpolazione	Fluida a 60°/s, nessun jitter rilevato

Metrica	Risultato
Stabilità broker 24h	Uptime 100% con restart policy Docker

10. Sicurezza del Sistema

10.1 Modello di Minaccia

Il sistema espone endpoint MQTT su internet tramite Ngrok. Il modello di sicurezza adottato affronta le principali superfici di attacco: accesso non autorizzato al broker, intercettazione del traffico, e iniezione di comandi malevoli.

10.2 Misure Implementate

Misura	Implementazione
Autenticazione MQTT	Username/password obbligatori, hash PBKDF2 (mosquitto_passwd)
Cifratura traffico web	TLS 1.3 su endpoint HTTPS/WSS Ngrok
Isolamento rete	Nessuna porta esposta direttamente sull'host Docker
Segmentazione container	Rete bridge ngrok-net isolata tra mqtt-broker e ngrok-agent
Validazione input ESP32	Clamping indice (0-3) e angolo (0-270°) nel firmware

10.3 Limitazioni e Raccomandazioni Future

- Le credenziali MQTT sono hardcoded nel firmware ESP32: in produzione, utilizzare NVS (Non-Volatile Storage) o un provisioning system
- Il tunnel Ngrok gratuito ha limiti di connessioni simultanee e non garantisce SLA: in produzione, utilizzare un server VPS con IP fisso
- Aggiungere autenticazione TLS client-side sul broker per i client IoT
- Implementare un sistema di autorizzazione per topic MQTT (ACL) per limitare i permessi per client

11. Conclusioni e Sviluppi Futuri

11.1 Risultati Raggiunti

Il progetto ha raggiunto tutti gli obiettivi prefissati: il braccio robotico è controllabile in tempo reale da qualsiasi dispositivo connesso a internet tramite l'applicazione Android o l'interfaccia web. L'interpolazione cinematica garantisce movimenti fluidi, e la simulazione 3D fornisce un feedback visivo immediato e accurato dello stato del braccio.

Le criticità architetturali affrontate durante lo sviluppo – dalla gestione dei CORS in WebView alle normali mancanti nei file CAD, dal disallineamento pivot-mesh alla mappatura degli indici JNI – hanno arricchito il know-how tecnico e portato a soluzioni robuste e riutilizzabili.

11.2 Sviluppi Futuri

11.2.1 Inverse Kinematics (IK)

Il passo naturale successivo è l'implementazione di un motore di Inverse Kinematics: l'operatore specificherebbe la posizione cartesiana desiderata dell'end-effector e il sistema calcolerebbe automaticamente gli angoli articolari necessari. Algoritmi come FABRIK o Jacobian pseudo-inverse sono adatti per questo tipo di configurazione planare.

11.2.2 Controllo a Traiettoria

Implementare la pianificazione di traiettorie multi-punto con profili di velocità trapezoidali (accelerazione/decelerazione graduale), migliorando la fluidità e riducendo lo stress meccanico durante movimenti complessi.

11.2.3 Feedback di Posizione

Aggiungere encoders o sensori di posizione angolare (potenziometri o encoder magnetici AS5600) per implementare un loop di controllo in retroazione, superando il controllo open-loop attuale.

11.2.4 Claw Funzionale

Ripristinare e aggiornare la configurazione della pinza (Servo 3), progettando un nuovo meccanismo di presa e reintegrando il link nell'albero cinematico della simulazione 3D.

11.2.5 Infrastruttura Produzione

Migrare l'infrastruttura broker da Ngrok a un VPS dedicato con IP fisso, certificato TLS Let's Encrypt e broker MQTT con QoS 1/2 per garantire la consegna affidabile dei comandi.

11.3 Conclusioni

Il progetto dimostra come sia possibile realizzare un sistema IoT completo e funzionale integrando tecnologie eterogenee: hardware embedded, protocolli di messaging distribuito, sviluppo mobile nativo, rendering 3D web-based e containerizzazione cloud. La separazione netta delle responsabilità tra i layer garantisce manutenibilità e scalabilità del sistema verso evoluzioni future.